PRODUCT REQUIREMENTS DOCUMENT

Scholarly LaTeX Editor

Codename: TBD

Document Version	0.1 — Initial Draft
Status	Pre-development
Last Updated	March 11, 2026
Confidentiality	Internal / Founder Only

Table of Contents

Table of Contents 2
1. Executive Summary 4
2. Problem Statement 4
  2.1 The LaTeX Tooling Gap 4
  2.2 The Administrative Burden in Large Projects 4
  2.3 Collaboration Friction 5
  2.4 Privacy and AI 5
3. Goals and Non-Goals 5
  3.1 Goals 5
  3.2 Non-Goals 6
4. Target Users 6
  4.1 Primary: Graduate Researchers 6
  4.2 Secondary: Research Groups and Academic Authors 6
  4.3 Tertiary: Technical Writers and STEM Students 6
5. Deployment Modes 6
6. Feature Specifications 7
  6.1 Core Editor — Document Tree and WYSIWYG Rendering 7
    6.1.1 Node Types 7
    6.1.2 Visual Rendering 7
    6.1.3 Automated Administrative Tasks 7
  6.2 Export 8
  6.3 Bibliography Management 8
  6.4 Asset Management and Version History 8
    6.4.1 Versioning Model 8
    6.4.2 Version UI 9
    6.4.3 Table Modes 9
  6.5 Real-Time Collaboration 9
  6.6 Annotation and Review System 9
    6.6.1 Inline Mode 9
    6.6.2 Issue Tracker Mode 9
    6.6.3 Annotation Data Model 10
  6.7 Project Timeline and Milestones 10
  6.8 Plugin System 10

6.8.1 Plugin Types 10
6.8.2 Bundled Core Plugins 11
6.8.3 Plugin Store 11
6.9 Template System 11
6.9.1 Template Contents 11
6.9.2 Versioning and Updates 11
6.9.3 Template Store 12
6.10 MCP Server — Document Context API 12
6.10.1 Exposed Tools 12
6.11 AI Features 12
6.11.1 Local AI (Offline Mode) 12
6.11.2 Cloud AI (Online Mode, Explicit Opt-In) 12
6.11.3 AI Is Never Forced 13
6.12 Security and Privacy 13
6.12.1 Encryption 13
6.12.2 Traffic Analysis Mitigation 13
6.12.3 Open Source and Self-Hosting 13
7. Feature Priority Summary 13
8. Technical Architecture Guidelines 14
8.1 Frontend 14
8.2 Backend (Online Mode) 15
8.3 MCP Server 15
8.4 LaTeX Export Engine 15
8.5 Plugin Architecture 15
9. Open Questions and Decisions Required 16
10. Success Metrics 16
10.1 Adoption 16
10.2 Collaboration 16
10.3 Export Quality 17
10.4 AI Feature Adoption 17
10.5 Privacy and Trust 17

1. Executive Summary

The Scholarly LaTeX Editor (working title TBD) is a modern, tree-based document authoring platform for academic researchers, graduate students, and technical writers. It unifies the typographic quality of LaTeX, the accessibility of a visual editor, the collaboration capabilities of Google Docs, and the reference management of Zotero or Mendeley — in a single application.

The core insight is that existing LaTeX editors treat the .tex file as the source of truth, forcing users to manage implementation details that have nothing to do with the intellectual work of writing. This product inverts that relationship: the document tree is the source of truth, and LaTeX is one of several export targets.

AI assistance is available as an opt-in enhancement and is never required. Privacy and user control are first-class design principles: the offline mode requires no account and no data ever leaves the device unless the user explicitly enables cloud features.

2. Problem Statement

2.1 The LaTeX Tooling Gap

LaTeX remains the standard for academic publishing in mathematics, physics, computer science, and engineering. Despite this, the tooling has not meaningfully advanced in decades. The dominant workflows are:

- Local editors (TeXShop, TeXstudio, VS Code + LaTeX Workshop): powerful but require manual project management, offer no real-time collaboration, and provide no visual feedback without compiling to PDF.
- Overleaf: adds cloud sync and basic collaboration but retains the flat .tex file model. Comments live in a dedicated column that fragments the screen. Reference management is rudimentary.
- Word + Mendeley/Zotero: accessible but produces documents that do not meet journal formatting requirements and requires manual reformatting for submission.

No existing option solves the full workflow. Researchers using LaTeX routinely maintain separate tools for writing (editor), reference management (Zotero/Mendeley), figure versioning (manual file naming conventions), collaboration feedback (email or PDF annotations), and project management (Notion, Asana, or ad hoc tracking).

2.2 The Administrative Burden in Large Projects

Large LaTeX projects such as PhD theses require significant non-writing overhead that consumes researcher time with no intellectual value:

- Managing \input chains across dozens of files
- Keeping \newcommand definitions consistent across chapters
- Maintaining acronym lists and ensuring consistent first-use expansion

- Manually updating figure paths when reorganising the project structure
- Keeping .bib files synchronised with in-text citations
- Tracking which version of a figure or table corresponds to which draft

A PhD student writing a 200-page thesis may spend 20-30% of their document-preparation time on these administrative tasks rather than on research writing.

2.3 Collaboration Friction

Academic writing is rarely solo. Advisor-student feedback cycles, co-authored papers, and journal revisions all require structured collaboration. Existing tools handle this poorly:

- Overleaf's comment system occupies a dedicated column, splitting the screen into four panels (navigator, editor, PDF, comments). This is unusable on standard laptop screens.
- Comments are not trackable as issues. There is no way to assign, prioritise, or monitor the resolution of reviewer feedback as a structured workflow.
- There is no timeline or milestone support for tracking submission deadlines against outstanding work.

2.4 Privacy and AI

Existing AI writing tools require sending document content to remote servers with no meaningful user control over what is transmitted or how it is used. For researchers working with unpublished results or under institutional data governance requirements, this is a hard blocker. No current tool offers AI assistance with genuine privacy controls.

3. Goals and Non-Goals

3.1 Goals

- Provide a visual, block-based editing experience that renders LaTeX elements in real time without requiring PDF compilation.
- Eliminate all administrative overhead in large LaTeX projects: file management, namespace consistency, label/ref resolution, asset paths, and bibliography sync are handled automatically.
- Integrate reference management as a first-class feature, replacing Zotero or Mendeley.
- Support real-time collaborative editing with a review and feedback system that does not sacrifice screen space.
- Provide annotation-driven project management: comments as issues, milestones, deadline tracking, and reminders integrated into the writing workflow.
- Offer AI writing assistance as a fully optional, explicitly opt-in feature with transparent disclosure of what data leaves the device.
- Deliver a complete, production-quality offline mode requiring no account and no network.
- Be open source, enabling institutional self-hosting and independent security auditing.
- Export to valid LaTeX and PDF without requiring the user to understand LaTeX internals.

3.2 Non-Goals

- Not a general-purpose word processor for non-academic documents.
- Not a spreadsheet or data analysis tool.
- Not a LaTeX IDE targeting power users who want full manual .tex control. Those users are well served by existing tools.
- Not an AI-first product. AI features are optional enhancements, not the core value proposition.
- Full support for all LaTeX packages is not a goal. Package support is delivered through a plugin architecture.

4. Target Users

4.1 Primary: Graduate Researchers

PhD and Master's students writing theses and journal papers. Key pain points: administrative overhead of large multi-file projects, advisor feedback cycles, figure versioning across drafts, and submission deadline management. High LaTeX familiarity but frustrated by tooling limitations.

4.2 Secondary: Research Groups and Academic Authors

Faculty, postdocs, and research teams co-authoring papers. Key pain points: real-time collaborative editing, structured reviewer comment tracking, consistent style enforcement across multiple contributors. Willingness to pay for tools that reduce publication workflow friction.

4.3 Tertiary: Technical Writers and STEM Students

Undergraduate students in technical fields, documentation writers, and anyone producing structured documents with mathematical content. Lower LaTeX familiarity — these users benefit most from the visual editing experience and the template system.

5. Deployment Modes

The product operates in two modes. The transition between them is explicit and always user-initiated. Offline mode is a complete, production-quality tool — not a degraded subset of online mode.

Capability	Offline Mode (default) | Online Mode (login required)
Storage	IndexedDB (local only) | IndexedDB (local cache) + encrypted cloud sync
Collaboration	Not available | Real-time CRDT multi-user editing
AI Features	Local LLM only, requires explicit setup | Cloud LLM available; off by default, per-project opt-in

Account Required	No | Yes
Data Leaving Device	Never (unless local AI explicitly configured) | Document content encrypted before any transmission; AI content sent only when explicitly opted in
Self-Hostable	N/A (fully local) | Yes — server is open source

6. Feature Specifications

6.1 Core Editor — Document Tree and WYSIWYG Rendering

The document is represented as a semantic tree. Every element is a typed node with a stable identity. The LaTeX source file and the rendered visual view are both projections of this tree; neither is the source of truth. This model enables automatic management of all administrative concerns that plague flat .tex projects.

6.1.1 Node Types

- Structural: Document, Chapter, Section, Subsection, Subsubsection, Appendix
- Content: Paragraph, Equation (inline and display), Figure, Table (editable and managed-asset modes), List (itemize/enumerate), Algorithm, Theorem/Lemma/Proof/Definition/Remark, Code block, Verbatim
- Metadata: Title, Authors, Abstract, Keywords, Affiliations, Submission Deadline
- Reference: Citation, Cross-reference, Footnote, Label
- Administrative (managed automatically, not directly exposed to user): Preamble, Command namespace, Acronym table, Bibliography store, Asset registry

6.1.2 Visual Rendering

- Equations rendered with KaTeX in real time as the user types LaTeX math syntax. No compile step required.
- Figures displayed as rendered images within the document flow, with caption and label visible.
- Tables rendered as formatted grids. Editable tables use a spreadsheet-like cell interface. Managed-asset tables display the LaTeX tabular output.
- Algorithm and theorem environments rendered with correct styling per the active document class.

6.1.3 Automated Administrative Tasks

- \newcommand namespace: commands defined once, enforced unique, available document-wide. Conflicts detected and surfaced to the user.
- Label/ref resolution: labels are node IDs, generated deterministically. Cross-references are always valid; broken refs are impossible.
- Multi-file serialisation: the tree is automatically serialised to .tex files, split by section depth and configurable file-size thresholds.

- Package inference: required packages are inferred from the node types in use and injected into the preamble at export time. Users never manually manage \usepackage.
- Asset path management: figures and tables are referenced by node ID; correct \includegraphics paths are generated on export regardless of how the project is structured.
- Acronym consistency: single definition per acronym node, first-use expansion tracked and enforced automatically.

6.2 Export

- LaTeX export: single .tex file or multi-file project structure, auto-managed. Output is valid, compilable LaTeX requiring no manual post-processing.
- PDF export: compiled via an in-browser WebAssembly LaTeX engine. Users do not need a local LaTeX installation for basic PDF output.
- Export configuration: document class, LaTeX engine (pdflatex/xelatex/lualatex), output naming, and preamble options are configurable per document.

6.3 Bibliography Management

The bibliography manager is a first-class integrated feature, not a plugin or external tool. It replaces Zotero and Mendeley for the academic writing workflow.

- Bibliography stored as a structured database of typed objects. \cite{} keys are foreign keys into this store; the key is managed automatically.
- Import: BibTeX (.bib files), DOI resolution, arXiv ID resolution, ISBN lookup.
- Export: BibTeX, biblatex, and common journal-specific formats.
- Duplicate detection and merge.
- Citation style preview: see how a reference appears in the document without compiling.
- Connected academic databases (online, user-authorised): Semantic Scholar, arXiv, INSPIRE-HEP, DOI resolver.
- AI-assisted citation suggestions (when AI enabled): based on document context, suggests relevant papers from the user's library or connected databases.

6.4 Asset Management and Version History

Figures and managed-asset tables are first-class versioned objects. Their content history is tracked separately from the document's CRDT operation log, which handles only structural and text changes.

6.4.1 Versioning Model

- Each asset node maintains an append-only version stack. The document tree stores a pointer to the active version ID; asset binaries never pass through the CRDT.
- Uploading a new version creates a new stack entry and updates the active pointer. Previous versions are retained and restorable.
- Version entries record: file, timestamp, author, and an optional change note.
- The CRDT log records 'active_version_id changed from vN to vM' as a single small operation.

6.4.2 Version UI

- Per-node version panel: full history, one-click restore to any previous version.
- Side-by-side visual comparison of any two versions.
- Collaborator activity feed notification when an asset is updated.

6.4.3 Table Modes

- Editable table: content as structured data in the tree (rows/columns/cells as text nodes). Edited in a spreadsheet-like UI. CRDT handles sync.
- Managed-asset table: content is an external file (CSV or LaTeX tabular blob, e.g. generated by an analysis script). Treated identically to figures for versioning. Suitable for tables that are regenerated programmatically.

6.5 Real-Time Collaboration

Available in online mode only, built on Yjs CRDT.

- Multiple users can edit simultaneously. Changes are merged without conflict using CRDT semantics. The server handles operation ordering and broadcast; merge logic runs client-side only.
- Presence indicators: online collaborators and their cursor positions are visible in real time.
- CRDT operation payloads are encrypted before transmission. The server sees operation metadata (ID, logical clock, parent IDs, type bucket) but not content.
- Operation type bucketing: operations are padded to fixed-size buckets by type to reduce information leakage from packet size analysis.
- Per-user undo/redo history derived from the CRDT operation log.

6.6 Annotation and Review System

Annotations use a unified data model. A comment is simultaneously an inline document annotation and a trackable project issue — the same object projected through two different UIs.

6.6.1 Inline Mode

- Annotations anchor to document tree nodes (not character offsets). They survive document restructuring — moving a section moves its comments.
- Rendered as small avatar badges in the document margin. Zero screen space consumed when not active.
- Clicking or hovering a badge expands a floating discussion card. Cards are draggable so they never obscure content the user needs to read.
- Reply, resolve, and re-open controls are accessible directly from the card.

6.6.2 Issue Tracker Mode

- Dedicated panel providing a project-management view of all annotations.
- List view with status (open, in progress, resolved, won't fix), priority (low, medium, high, blocking), assignees, labels, and due dates.

- Filter by section, assignee, status, priority, or label.
- Clicking any row jumps the document to the anchored node and expands the inline card.
- Inline and tracker modes are always synchronised — state changes in one reflect immediately in the other.

6.6.3 Annotation Data Model

- Each annotation: discussion thread with @mentions, assignees (multiple), labels, priority, due date, and links to related annotations.
- Submission deadline awareness: the editor surfaces counts of open blocking annotations and overdue items against the document's submission deadline.

6.7 Project Timeline and Milestones

Each document has an associated timeline that integrates with the annotation system and the document's submission deadline metadata field.

- Milestones: named, dated checkpoints (e.g. 'Send draft to advisor', 'Submit to ICHEP', 'Final thesis deposit'). Visible as markers on the timeline and referenceable from annotations.
- Submission deadline: a top-level document metadata field. Drives deadline-aware features throughout the editor.
- Timeline view: Gantt-style visual showing milestones, their status, and counts of open blocking annotations per milestone.
- Reminders: configurable notifications (on document open, N days before milestone) for approaching deadlines with unresolved blocking issues.
- Milestone status: pending, in progress, complete, or missed. Provides a project health overview at a glance.

6.8 Plugin System

LaTeX package support is delivered through plugins. The core editor ships bundled support for the most common academic packages. Everything else is community-contributed via the plugin store.

6.8.1 Plugin Types

- Rendering plugins: packages that produce visual output requiring a custom block renderer (tikz, pgfplots, chemfig, circuitikz). Register React components that render their environments in the WYSIWYG view.
- Semantic plugins: packages that define new document structure and new node types (algorithm2e, listings, tcolorbox, amsthm). Register new node types and LaTeX serialisers.
- Namespace plugins: packages that add inline commands or symbols (siunitx, physics, mhchem). Register new \commands and their rendering behaviour.
- Export-only plugins: packages affecting PDF output only, with no editor-side visual behaviour (hyperref, microtype, geometry). Injected into preamble at export time at zero editor complexity cost.

6.8.2 Bundled Core Plugins

- Math: amsmath, amssymb, mathtools
- Figures: graphicx, subcaption, wrapfig
- Tables: booktabs, longtable, multirow
- Algorithms: algorithm2e
- Code: listings, minted
- Diagrams: tikz, pgfplots
- Units: siunitx
- Chemistry: mhchem
- Export-only: hyperref, microtype, geometry, cleveref

6.8.3 Plugin Store

- Community store for packages not covered by bundled plugins. Plugins are versioned; a document records required plugin versions.
- Verification tier: plugins published by journal publishers or known institutions are badged as verified.
- Plugins extend the MCP server: each plugin can register a contextSummary function that controls how its node types are represented in LLM context windows.

6.9 Template System

Templates are versioned, frozen project snapshots. Creating a project from a template is analogous to instantiating a Docker container from an image: the template is immutable, and the project diverges from it immediately on creation.

6.9.1 Template Contents

- Document tree skeleton: predefined sections, placeholder nodes, sample figures and tables showing correct formatting for the target journal or document type.
- Plugin manifest: exact plugin versions required. Plugins are auto-installed on instantiation.
- Document class and preamble configuration: journal class file, margin and font settings.
- Asset stubs: placeholder figures, institution logos (thesis templates), sample tables.
- Bibliography style: the .bst or biblatex style required by the target venue.
- Setup script: interactive questionnaire run once on instantiation. Populates metadata nodes (title, authors, institution, submission date). Answers propagate automatically to all parts of the tree that use them — no manual placeholder replacement.

6.9.2 Versioning and Updates

- Templates follow a layered model — base layer (document class, plugin manifest), style layer (formatting), content layer (user's writing). Updates from template publishers touch only base and style layers; user content is never affected.
- Projects record the template version they were instantiated from. Updates can be applied selectively.

6.9.3 Template Store

- Official templates: major journals (Physical Review Letters, IEEE, ACM, Nature, JHEP, arXiv preprint), common thesis formats.
- Institutional templates: universities publish thesis/dissertation templates findable by institution name.
- Community templates: user-contributed, with a verification tier for official publisher submissions.

6.10 MCP Server — Document Context API

The document tree is exposed as a Model Context Protocol server. This enables both the editor's own AI features and external tools (such as Continue in VS Code) to consume rich structured document context rather than parsing raw .tex files.

6.10.1 Exposed Tools

- get_node(id): content and metadata of a specific tree node.
- get_context(cursor_node_id, token_budget): assembles a context string by traversing the tree outward from the cursor. Includes command namespace and acronym table automatically.
- resolve_ref(label): returns the node containing the given \label, enabling cross-reference queries without file search.
- search_bib(query): returns ranked citation candidates from the bibliography store.
- get_namespace(): all \newcommand definitions and acronym entries as a compact table.
- list_assets(): all figures and tables with captions, labels, and active version metadata.
- get_timeline(): milestones, deadlines, and open blocking annotations for project-status queries.

The MCP server runs locally in offline mode. In online mode it is optionally accessible to authorised external tools via the user's session.

6.11 AI Features

All AI features are optional. The editor is fully functional without any AI. AI is disabled by default and requires explicit user action to enable at each level.

6.11.1 Local AI (Offline Mode)

- Local LLM support via the user's own model runtime (e.g. Ollama, llama.cpp). Setup requires an explicit approval step — not automatic or silent.
- FIM (fill-in-the-middle) inline completion: context assembled by the MCP server, delivered to the model in Qwen FIM format. No document content leaves the device.

6.11.2 Cloud AI (Online Mode, Explicit Opt-In)

- Cloud LLM via configurable API endpoint (Anthropic, OpenAI, or user-provided). Off by default.
- Enabled per-project via an explicit toggle with a clear disclosure statement: 'Enabling AI will send document content to [provider name].'
- AI-assisted features when enabled: inline FIM completion, citation suggestions, annotation drafting, section summarisation.

6.11.3 AI Is Never Forced

- No core workflow depends on AI being enabled. Features that rely on AI are clearly labelled as optional enhancements.
- The product must never degrade in usefulness when AI is disabled.

6.12 Security and Privacy

6.12.1 Encryption

- All client-server communication over HTTPS.
- Document content encrypted at rest on the server using a key derived client-side via PBKDF2. The server never has access to the plaintext encryption key.
- CRDT operation payloads encrypted before transmission. The server sees only metadata required for ordering (operation ID, logical clock, parent IDs, type bucket), not content.
- Collaboration key exchange: document encryption key shared with collaborators by encrypting it with their public key (X25519). The server brokers the exchange without ever seeing the plaintext key.

6.12.2 Traffic Analysis Mitigation

- CRDT operations padded to fixed-size buckets by operation type. Reduces information leakage from packet size analysis from ~32 bits of entropy to ~2 bits.
- Product documentation is transparent about residual metadata leakage (operation timing and frequency) and the practical limits of protection without unacceptable latency overhead.

6.12.3 Open Source and Self-Hosting

- Server and client are fully open source. Users and institutions can audit the implementation and verify stated privacy properties.
- The sync server is self-hostable with minimal infrastructure. Universities and research groups can operate their own instances.
- Self-hosted deployments can disable cloud AI endpoints, enforcing local-only AI for all project members.

7. Feature Priority Summary

Feature	Description	Priority	Mode
Document Tree + WYSIWYG	Core editing model; all features depend on this	P0 — MVP	Both
LaTeX / PDF Export	Primary output target; required for usefulness	P0 — MVP	Both
Bibliography Manager	Replaces Zotero/Mendeley; key differentiator	P0 — MVP	Both
Offline Mode	Zero-dependency local use; foundation of trust model	P0 — MVP	Offline
Core Plugins	Required for standard LaTeX environments to render	P0 — MVP	Both
Template System	Critical for journal submission workflow; drives adoption	P1	Both
Real-Time Collaboration	Core online feature; requires CRDT + sync server	P1	Online
Asset Versioning	High value for iterative research	P1	Both
Annotation + Issue Tracker	Replaces email feedback; strong differentiator vs Overleaf	P1	Both
MCP Server	Enables AI and external tooling integration	P1	Both
Project Timeline + Milestones	Reduces need for external PM tools; reinforces retention	P2	Both
AI — Local LLM	Differentiator for privacy-conscious users; requires MCP	P2	Offline
AI — Cloud LLM	Broadens AI accessibility; requires explicit opt-in	P2	Online
Community Plugin Store	Long-term ecosystem building; not required for launch	P3	Both
Community Template Store	Long-term ecosystem; official templates sufficient for launch	P3	Both
Institutional Templates	Enterprise/university sales; post-launch	P3	Online

8. Technical Architecture Guidelines

This section provides high-level implementation direction. A detailed plan with task-level specifications, dependency mapping, and ticket templates will be produced as a separate document.

8.1 Frontend

- React application. Targets both browser (web app) and optionally Electron (offline desktop packaging).

- Block editor built on ProseMirror or Lexical. Custom node views implemented as React components per node type.
- KaTeX for real-time equation rendering (client-side, no server dependency).
- IndexedDB via Dexie.js as the primary local store for the document tree, bibliography, and asset metadata.
- Yjs for CRDT state management. Yjs update payloads are encrypted before any network call.

8.2 Backend (Online Mode)

- Minimal sync server: relays encrypted Yjs updates, stores encrypted document blobs, manages user auth and key exchange. No document parsing, no merge logic, zero plaintext document access.
- WebSocket endpoint for Yjs relay (y-websocket protocol compatible).
- REST API for encrypted blob storage, user management, and asset upload.
- Auth: JWT with short expiry and refresh tokens. Public key stored per user for key exchange.
- Storage: S3-compatible object store for encrypted asset blobs; PostgreSQL or SQLite for user and project metadata.

8.3 MCP Server

- Runs as a local process in offline mode (Node.js or Python, communicating with the React app via localhost IPC).
- Stateless with respect to the document — reads from the same IndexedDB store the editor uses.
- In online mode, optionally accessible to authorised external tools via session token.

8.4 LaTeX Export Engine

- Tree-walker that emits LaTeX: implemented as a pure function (tree input, string output). No LaTeX parsing in the export path.
- Multi-file export: sections above a configurable depth are emitted as separate .tex files with \input references in the root file.
- PDF compilation: WebAssembly LaTeX engine (SwiftLaTeX or texlive.js) for in-browser compilation. Server-side fallback available for complex documents or packages not supported by the WASM engine.

8.5 Plugin Architecture

- Plugins are ES modules loaded at runtime, each exporting a manifest conforming to the EditorPlugin interface.
- Core bundled plugins compiled into the main application bundle. Community plugins loaded on demand.
- Plugins are sandboxed via module boundaries and can access document state only through the registered node type API.

9. Open Questions and Decisions Required

Item	Question / Decision Required
Product Name	TBD. Must be decided before any public-facing work.
Offline Storage Limit	IndexedDB has browser-enforced limits (~1 GB typical). Is a native file system fallback required for large projects with many high-resolution figures?
LaTeX Compilation Engine	SwiftLaTeX (WASM, in-browser) vs server-side compilation. WASM preferred for privacy but has compatibility gaps with some packages.
Plugin Sandboxing Strictness	ES module isolation is lightweight but not security-grade. Are community plugins required to pass a review process before publication?
Collaboration Auth	Who can invite collaborators? Document owner only, or any project member? What happens when a collaborator is removed — are their pending CRDT operations applied?
Pricing Model	Offline mode free forever. Online mode: freemium with storage limit on free tier, or subscription? Institutional licensing tier?
.tex Import at MVP	Importing existing .tex projects into the tree is the hardest engineering problem. Is this required for MVP or deferred to post-launch?
Mobile Support	Mobile-responsive web UI at launch, or desktop-first?
AI Provider Support	Which cloud AI providers are supported at launch? Anthropic only, or multi-provider from day one?
Annotation Notifications	In-app notifications only, or email/push for annotation assignments and milestone reminders?

10. Success Metrics

10.1 Adoption

- Monthly Active Users (MAU), offline and online combined.
- Projects created per user per month.
- 30-day and 90-day user retention rates.
- Template instantiations as a proxy for new project starts.

10.2 Collaboration

- Percentage of online projects with two or more active collaborators.
- Average annotations per collaborative project.
- Annotation resolution rate: percentage of opened annotations resolved within 14 days.

10.3 Export Quality

- Percentage of PDF exports that compile without error on first attempt.
- User-reported export issues per 1,000 exports.

10.4 AI Feature Adoption

- Percentage of users who have enabled AI features. High adoption would indicate the opt-in disclosure is insufficiently prominent.
- AI completion acceptance rate: accepted suggestions divided by total suggestions shown.

10.5 Privacy and Trust

- Zero reported incidents of unauthorised document content access.
- Percentage of power users running self-hosted server instances as a proxy for institutional trust.

This PRD reflects design discussions as of March 11, 2026. A detailed implementation plan with task-level specifications, dependency mapping, and Linear / Asana ticket templates will be produced as a separate document.